Advanced Empirical Finance: Topics and Data Science

Welcome, setup and financial data

Stefan Voigt

University of Copenhagen and Danish Finance Institute (DFI)

Spring 2027

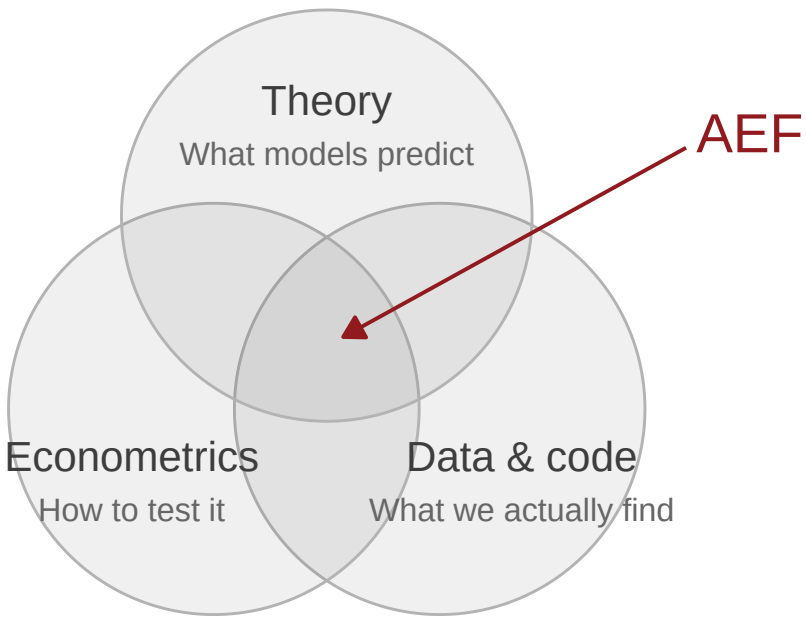
Welcome!

Why I teach this course

1. **PhD at VGSF**
   Replicating published work is far more cumbersome than it should be
2. **Decisions with large data**
   High-frequency data: 100 billion observations, computed on HPC clusters
3. **Tidy Finance**
   Open-source code for empirical finance, with Christoph Scheuch, my office colleague at VGSF
4. **Reproducibility**
   How the choices of researchers change their results
5. **Today and next**
   Machine learning and LLMs in research: what I enjoy and want to share

What AEF is about

This course is about doing empirical finance: working with real data, from start to finish.



- Empirical finance is finance measured against reality
- Financial markets provide vast data to inform financial decisions

(How) can state-of-the-art methods improve financial decision making?

One thread through the course

From portfolio choice to pricing

February to mid-March

The efficient portfolio and its estimation error; shrinkage and transaction costs; the SDF and the CAPM; factors, sorts and reproducibility

Machine learning and the SDF

Mid-March to April

Overfitting and out-of-sample evaluation; shrinking the SDF; latent factors; trees, forests and neural networks on the factor zoo

Volatility, risk and uncertainty

End of April and May

Volatility and covariances from high-frequency data; covariance forecasts for portfolio choice

How the course works

Team and contact

SV

Stefan Voigt, lecturer

stefan.voigt@econ.ku.dk

voigtstefan.me

NZ

Nikolai Zakarias Zwisler, teaching assistant

xjb522@econ.ku.dk

You

You and your peers

Teams of up to three, and the people who read your code in peer feedback: connect and help each other

> Questions about content go to the **Absalon discussion board**, so everyone benefits from the answer

Every week

- **Lectures:** Mondays 8–10 in even weeks (CSS 1-1-18) and Thursdays 13–15 (CSS 18-01-11): methods, theory and recent papers
- **Exercises:** weekly, in R or Python, built on **Tidy Finance**; the solutions are in the book
- **Exercise classes:** Mondays 8–10 in odd weeks (CSS 2-2-36): work on the exercises and ask for help

> **Most of your time goes into the exercises.** Work through them on your own. Use AI, but make sure you know what is going on: the lectures build on your work on the exercises.

The learning curve is steep: ask early, me directly, on Absalon or in the exercise classes

Mandatory assignments

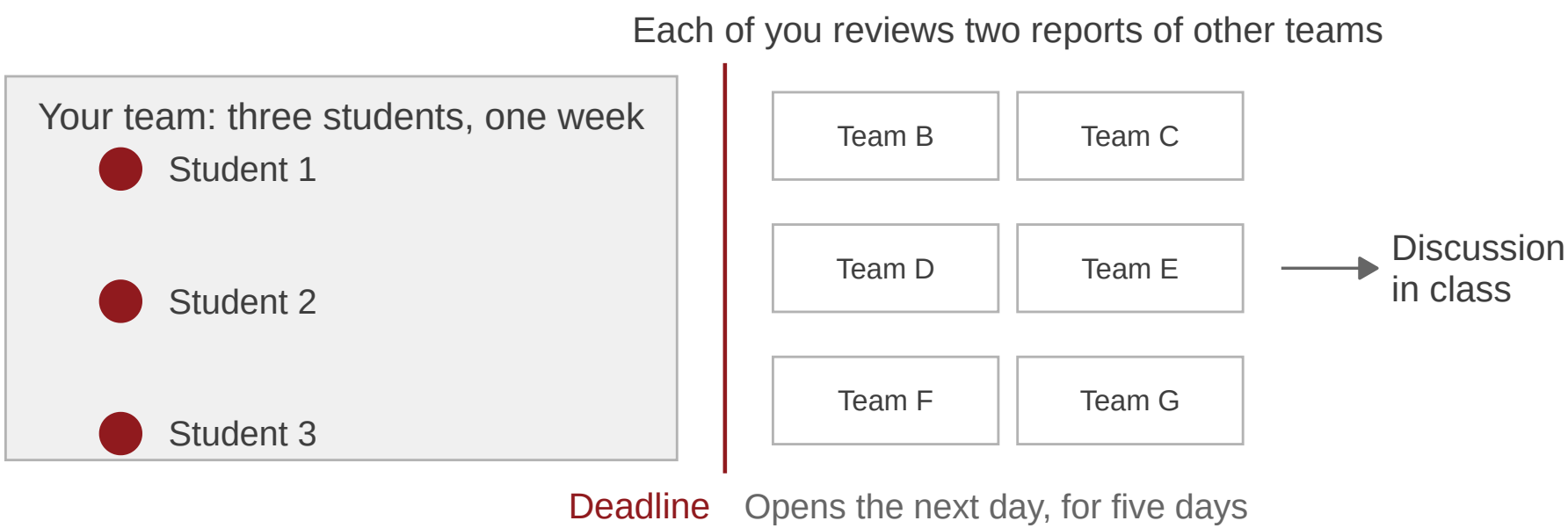
	Topic	Date
1	Portfolio choice	11–18 March
2	Machine learning in asset pricing	19–26 April
3	Volatility and covariances from high-frequency data	3–10 May

- One week each; the assignments and the data come through Absalon
- Hand in one `.qmd` file and the PDF rendered from it: at most 7 pages, 12 pt, in English, from the templates

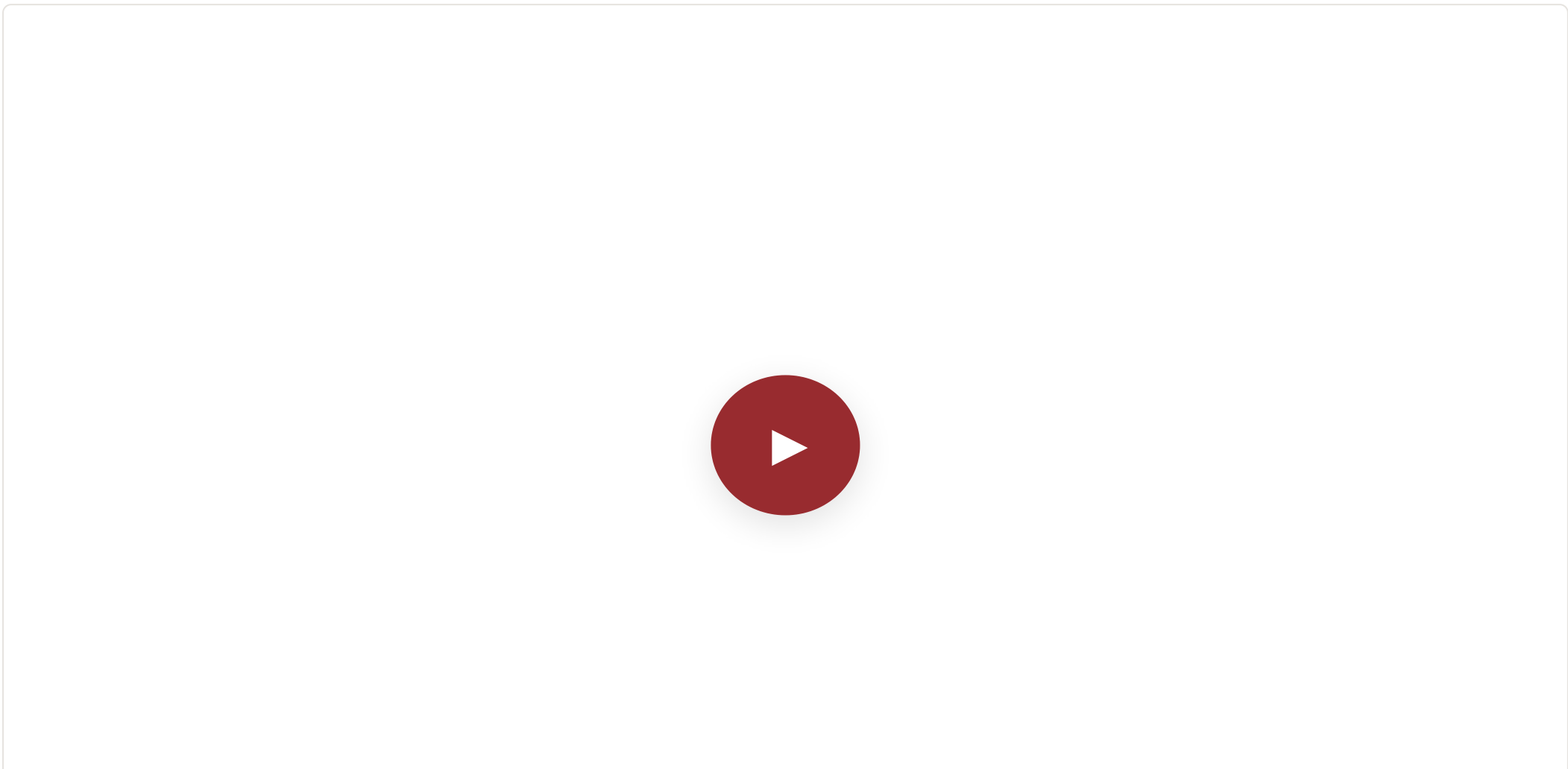
> **The deal:** use AI as much as you like, for code, text and review, and say how you used it. In return, you answer for every line, and we grade by a high bar: the **assignment quality criteria**.

Peer feedback

- Peer feedback opens **one day** after the deadline and stays open for **five** days
- Each of you gives feedback on **two** submissions of other teams
- Start by reproducing their report: does it render from their files, and do the numbers match?
- We discuss the feedback in class: you see how others solved the same problem



The trading game



The 60-second introduction on the homepage: **aef-course.ku.dk/trading-game**

The trading game: what matters

- The game is there to make you decide, and to discuss decisions
- Rules, tools and the dry run: **aef-course.ku.dk/trading-game**
- **Conference session** after the last lecture: the teams with the best out-of-sample performance present their approach, and other students discuss it, in front of faculty and quantitative traders
- **A jury** picks the best team, on investment performance **and** presentation

10,000 DKK for the winning team

How you are assessed

Continuous assessment: what you hand in and present during the semester counts

- **Three mandatory assignments**, in teams of up to three
- **Peer feedback** on the assignments of other teams, individually
- **Discussion sessions:** presentations and discussions

Who are you?

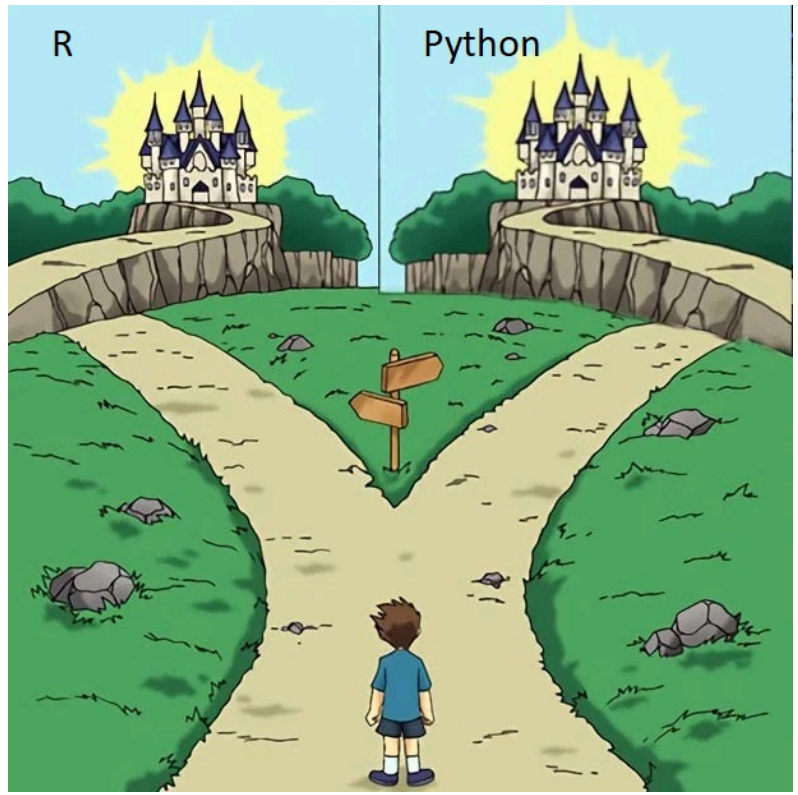
A short anonymous survey in the first lecture:

- Who are you: master's, exchange, or PhD student?
- How good are you at R, Python, and AI tools for coding?
- Which tools have you used?
- Have you completed the technical prerequisites?
- What do you mainly want to take away from this course?

Your setup

R or Python?

- Both are open source, have large communities and are standard in finance
- Pick one; every exercise and every solution exists in both
- **Tidy coding** matters more than the language: clean code in either is readable for everyone
- You give feedback on R and Python submissions: with tidy code, you understand your peers' code even in the other language



The toolbox

Tool	What it does
R or Python	The language you write your analysis in
Positron	The editor: code, console, data viewer, Git and AI assistant in one place
Quarto	Weaves text and code into one report: your hand-ins are Quarto files rendered to PDF
uv and uvr	Install Python, R and exactly the course's package versions
Git and GitHub	Version control and your team's repository

Financial data

Where the data come from

- **CRSP** (Center for Research in Security Prices): every US common stock since 1926, from WRDS. Licensed: you get it through Absalon
- Everything lives as parquet files in the `data` folder of the course repository, as in the Tidy Finance book

File	Content
`crsp_monthly.parquet`	CRSP monthly returns, market capitalization, exchange, industry (from Absalon)
`factors_ff3_monthly.parquet`	Fama-French three factors and the risk-free rate
`industries_ff_monthly.parquet`	Ten industry portfolios
`macro_predictors.parquet`	Welch and Goyal (2008) predictors
`cpi_monthly.parquet`	Consumer price index

Reading the data

R

```r
crsp_monthly <- read_parquet("data/crsp_monthly.parquet")
cpi_monthly <- read_parquet("data/cpi_monthly.parquet")
```

- The CRSP file is licensed: never commit it, never upload it, never paste it into an AI tool. Git ignores it in the course repository
- Paths start at the repository root (`data/...`), so your code runs on your teammates' computers too

Reading the data

Python

```
crsp_monthly = pl.read_parquet("data/crsp_monthly.parquet")
cpi_monthly = pl.read_parquet("data/cpi_monthly.parquet")
```

- The CRSP file is licensed: never commit it, never upload it, never paste it into an AI tool. Git ignores it in the course repository
- Paths start at the repository root (`data/...`), so your code runs on your teammates' computers too

Tidy coding: the essentials

Tidy data and the pipe

- **Tidy data:** each variable is a column, each observation a row, each value a cell. CRSP is tidy: one row per stock and month
- **One verb per step, chained:** `|>` in R, method chaining in Python. The code reads like the analysis
- Names say what a column holds (`trading_volume_usd`, not `tmp2`); the **quality criteria** have the rest

R

```
crsp_monthly |>
  filter(date == max(date)) |>
  count(exchange)
```

```
# A data frame: 3 × 2
  exchange     n
  <chr>    <int>
1 AMEX       146
2 NASDAQ    2298
3 NYSE      1241
```

Tidy data and the pipe

- **Tidy data:** each variable is a column, each observation a row, each value a cell. CRSP is tidy: one row per stock and month
- **One verb per step, chained:** `|>` in R, method chaining in Python. The code reads like the analysis
- Names say what a column holds (`trading_volume_usd`, not `tmp2`); the **quality criteria** have the rest

Python

```
(
    crsp_monthly.filter(pl.col("date") == pl.col("date").max())
    .group_by("exchange")
    .len()
    .sort("exchange")
)
```

shape: (3, 2)

exchange	len
str	u32
"AMEX"	146
"NASDAQ"	2298
"NYSE"	1241

A first case study: daily returns

R

```r
prices <- download_data(
  domain = "Stock Prices",
  symbols = symbols,
  start_date = start_date,
  end_date = end_date
)

returns <- prices |>
  arrange(symbol, date) |>
  mutate(ret = adjusted_close / lag(adjusted_close) - 1, .by = symbol) |>
  select(symbol, date, ret) |>
  drop_na(ret)
```

Each line is one step of the analysis: sort, compute the return of each stock, keep what you need, drop the first day

A first case study: daily returns

Python

```python
prices = tf.download_data(
    domain="Stock Prices",
    symbols=symbols,
    start_date=start_date,
    end_date=end_date,
)

returns = (
    prices.sort("symbol", "date")
    .with_columns(ret=pl.col("adjusted_close").pct_change().over("symbol"))
    .select("symbol", "date", "ret")
    .drop_nulls("ret")
)
```

Each line is one step of the analysis: sort, compute the return of each stock, keep what you need, drop the first day

Summaries by group

R

```
returns |>
  summarize(
    mean = mean(ret),
    sd = sd(ret),
    min = min(ret),
    max = max(ret),
    .by = symbol
  )
```

```
# A tibble: 2 × 5
  symbol     mean     sd    min   max
  <chr>     <dbl>  <dbl>  <dbl> <dbl>
1 AAPL   0.00119  0.0243 -0.519 0.153
2 MSFT   0.000578 0.0189 -0.156 0.196
```

`group_by` and `summarize` (or `agg`) answer many questions in empirical finance

Summaries by group

Python

```
returns.group_by("symbol").agg(
    mean=pl.col("ret").mean(),
    sd=pl.col("ret").std(),
    min=pl.col("ret").min(),
    max=pl.col("ret").max(),
)
```

shape: (2, 5)

symbol	mean	sd	min	max
str	f64	f64	f64	f64
"AAPL"	0.001191	0.024263	-0.518692	0.153289
"MSFT"	0.000578	0.018901	-0.155978	0.195653

`group_by` and `summarize` (or `agg`) answer many questions in empirical finance

The grammar of graphics

Data, a geometric shape, and aesthetics that map variables to it: `ggplot2` in R, `plotnine` in Python

R

```
prices |>
  ggplot(aes(x = date, y = adjusted_close)) +
  geom_line() +
  facet_wrap(~symbol, scales = "free_y") +
  labs(
    x = NULL, y = "USD",
    title = "Adjusted closing prices of Apple and Microsoft, daily"
  ) +
  theme_bw()
```

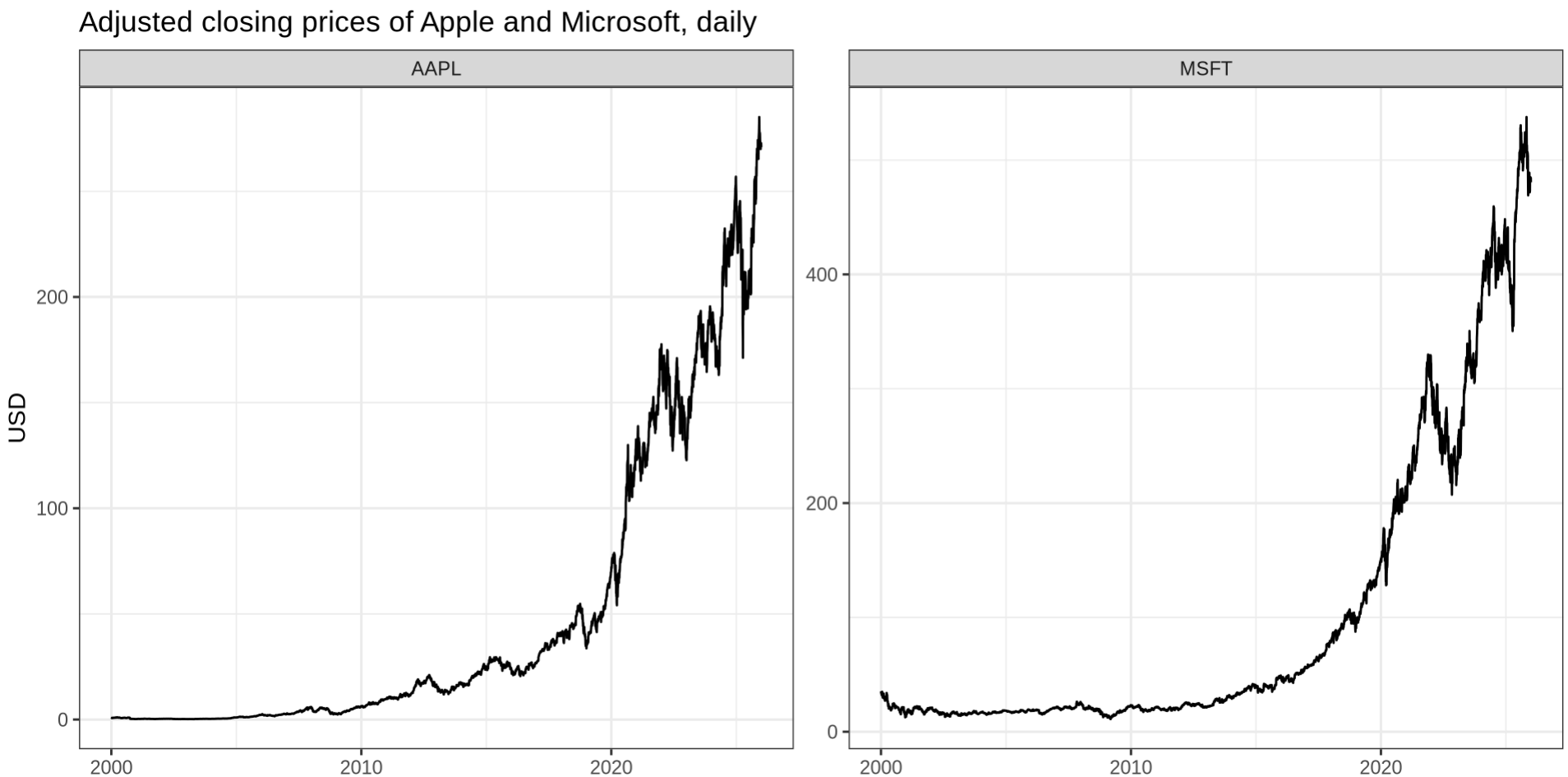
The grammar of graphics

Data, a geometric shape, and aesthetics that map variables to it: `ggplot2` in R, `plotnine` in Python

Python

```
(
    ggplot(prices, aes(x="date", y="adjusted_close"))
    + geom_line()
    + facet_wrap("symbol", scales="free_y")
    + labs(
        x="", y="USD",
        title="Adjusted closing prices of Apple and Microsoft, daily",
    )
    + theme_bw()
)
```

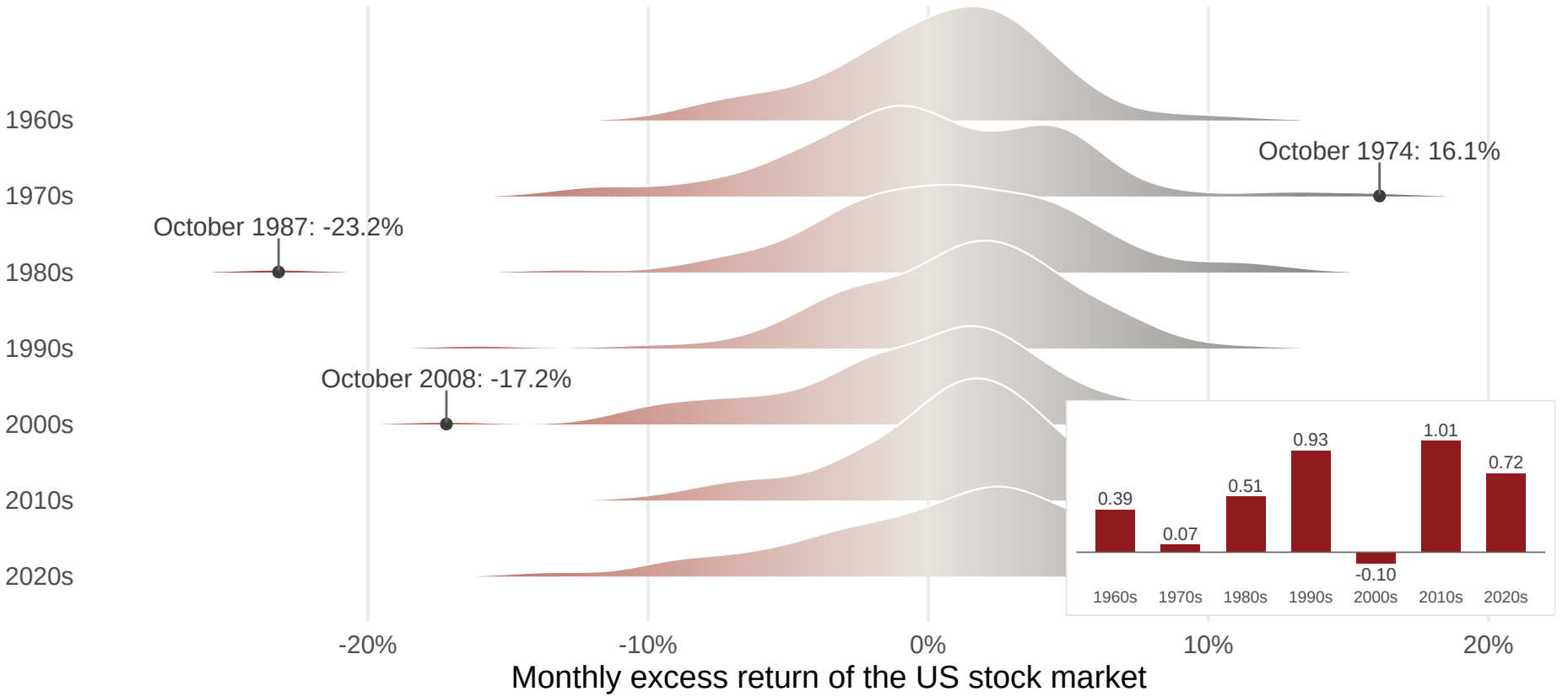
The grammar of graphics



What the grammar of graphics can do

Six and a half decades of the US stock market

Distribution of monthly excess returns by decade; inset: annualized Sharpe ratio



Data: Kenneth R. French data library

The same layers as on the slide before, plus two packages: ggridges and patchwork

Where to get help

- **The homepage:** aef-course.ku.dk has the lecture plan, the exercises, the trading game and the guides
- **Absalon discussion board:** questions about content and code, so everyone benefits from the answer
- **Exercise classes:** Mondays 8–10 in odd weeks (CSS 2-2-36)
- **Me directly:** stefan.voigt@econ.ku.dk